

TRANSACT-SQL Stored Procedure & Function

朱克剛



變數宣告

- 變數使用前必須要宣告，前面一定要加 @

```
declare @n int = 0  
declare @text nvarchar(50) = null
```

- 賦值

```
set @n = 10
```

- 將查詢結果放到變數中

```
select @text = cname from UserInfo where uid = 'A01'
```

判斷式 – if ... elseif ... else

- 壽星當月買東西有打折
- 根據帳號，如果該帳號是當月壽星，輸出 0.8，否則輸出 1.0

```
declare @uid nvarchar(20) = 'A01'
if month(getdate()) = (
    select month(birthday) from UserInfo where uid = @uid
)
begin
    select 0.8 as discount
end
else
begin
    select 1.0 as discount
end
```

注意是一個「=」號

迴圈 – while

- 在 UserInfo 中插入 10 筆資料
- 迴圈中可以使用 break 與 continue 做流程控制

```
declare @n int = 10
while @n > 0
begin
    insert into UserInfo (uid, cname) values (
        concat('X', @n),
        concat('X', @n)
    )
    set @n = @n - 1
end
```

系統變數 @@error

- @@error 為最近一筆 SQL 指令執行結果，若發生錯誤時，@@error 不等於 0

```
insert into UserInfo (uid) values ('B01')
if @@error > 0
begin
    print('insert data fail')
end
else
begin
    print('insert data success')
end
```

Try...Catch

- 攔截錯誤，例如下面的 SQL 指令會造成 PK 重複的錯誤，然後被 catch 攔截並處理

```
begin try
    insert into UserInfo (uid) values ('A01')
end try
begin catch
    print(error_number()) -- 相當於 @@error
    print(error_message())
end catch
```

預存程序

預存程序與函數

- 使用預存程序（ stored procedure ）或函數（ stored function ）將 T-SQL 程式碼儲存在資料庫中，需要使用時呼叫他們
- 程序：執行結果不透過 return 方式傳回
- 函數：執行結果以 return 方式傳回
- SQL server 有許多內建的預存程序，全部以 sp_ 開頭
- 查詢有哪些資料庫

exec sp_databases

- 查詢有哪個資料庫有哪些資料表

exec sp_tables @table_qualifier='AddressBook', @table_owner='dbo'

使用者登入的預存程序

- 建立一個預存程序用來檢查使用者登入的帳號密碼是否正確

```
create proc login
    @uid nvarchar(20),
    @password nvarchar(64)
as
begin
    select count(*) as result
    from UserInfo
    where uid = @uid and password = @password
end
```

- 執行

```
exec login 'A01', '1234'
```

練習

- 建立一個使用者註冊的預存程序
- 輸入參數：uid、password 與 address
- 不論 address 存不存在，一律新增一筆
- Password 需以 sha256 方式加密儲存於資料庫

預存程序傳回資料方式

- 資料集 (record set 或 result set) ，只能傳回一個資料集

```
begin  
    select 1 as status  
end
```

- Output 參數
- 透過 return ，但只能傳回一個整數數字

透過 Output 參數

- 使用 output 參數傳回查詢結果

```
create proc getname
    @uid nvarchar(20),
    @cname nvarchar(50) output
as
begin
    select @cname = cname from UserInfo where uid = @uid
end
```

- 執行方式

```
declare @result nvarchar(50)
exec getname 'A01', @result output
select @result
```

透過 Return 傳回狀態

- 只能傳回整數型態，用來表示執行狀態

```
create proc new_login  
    @uid nvarchar(20),  
    @password nvarchar(64)  
as  
begin  
    declare @status int  
    select @status = count(*) from UserInfo  
    where uid = @uid and password = @password  
    return @status  
end
```

- 執行方式

```
declare @result int  
exec @result = new_login 'A01', '12345'  
select @result
```

練習

- 打算設計一個玩猜數字比大小的遊戲，玩法是電腦隨機產生一個數字然後人來猜。如果猜錯，會傳回太大或太小的訊息，如果猜對會顯示猜對了。根據ER，玩家資訊使用UserInfo資料表，除此之外還需要新增哪些表？
- 建立預存程序當作遊戲介面

自訂函數

自訂函數

- 有時也稱為 Stored Function (預存函數)
- 建立一個加法函數

```
create function myadd(@a int, @b int)  
returns int  
as  
begin  
    return @a + @b  
end
```

- 呼叫時，函數名稱前要加上 schema

```
select dbo.myadd(5, 3)
```

行內資料集函數

- 函數傳回型態為 table
- 內容只能有一行 return 指令，因此沒有 BEGIN END
- 類似 View，但與 View 的差別在於可以輸入參數

```
create function show_userinfo(@uid nvarchar(20))  
returns table  
as  
return (  
    select * from UserInfo where uid = @uid  
)
```

- 呼叫方式

```
select * from dbo.show_userinfo('A01')
```

多敘述資料集函數

- 傳回的資料表內容可以先「組裝」完再傳回

```
create function demo_userinfo(@uid nvarchar(20))
returns @t table (uid nvarchar(20), cname nvarchar(20))
as
begin
    insert @t select uid, cname from UserInfo
    insert @t select @uid, '測試帳號'
    return
end
```

補充 - 暫存表

- SQL Server 有三種類型的暫存表，儲存在 tempdb 中

- #Table
- ##Table
- @table

- #Table

- 建立者擁有，連線中斷後會消失

- ##Table

- 全域，所有使用者斷線後會消失

- @table

- 資料型態為 Table 的變數，指令執行完就消失

```
declare @table Table
```

```
(  
    product varchar(50),  
    price int  
)
```

```
insert into @table values ('鉛筆', 10)
```

```
insert into @table values ('原子筆', 20)
```

```
select * from @table
```

CURSOR

用途

- 使用迴圈，對查詢的資料逐筆處理
- 當一個 SQL 指令無法完成的工作，通常就需要使用 cursor
- 使用時的程序

建立 → **declare c cursor for** 某個查詢指令

開啟 → **open c**

關閉 → **close c**

移除 → **deallocate c**

範例：使用 cursor 計算帳單金額總和

```
begin
  declare c cursor for select fee from Bill
  declare @fee int
  declare @sum int = 0
  open c
  fetch c into @fee
  while (@@fetch_status = 0)
  begin
    set @sum = @sum + @fee
    fetch c into @fee
  end
  select @sum as '帳單金額總和'
  close c
  deallocate c
end
```

cursor

cursor

cursor

fee
250
300
100

練習

- 使用 cursor 將沒有設定密碼的使用者，設定密碼為帳號後兩碼加上生日的月日
- 例如 A02 的生日為 1998-10-02，密碼改為 021002

可改變資料內容的 Cursor

- 透過 cursor 更新資料時，可以先宣告該 cursor 為 for update

```
declare c cursor for 某個查詢指令 for update
```

```
while (@@fetch_status = 0)  
begin  
    某個更新指令 where current of c  
end
```

- 當然也可以透過 fetch 得到 PK，然後透過 PK 來更新資料